

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**  
**ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА**

Факультет математики, фізики та інформаційних технологій

Кафедра математичного забезпечення комп'ютерних систем

**ЗВІТ**

з освітнього компоненту «Програмування»

**Лабораторна робота №4**

**на тему: СИСТЕМА УПРАВЛІННЯ ПЕРСОНАЛОМ**  
**(ІЄРАРХІЯ КЛАСІВ PERSON)**

Студента 1 курсу, денна форма навчання  
спеціальність F7 «Комп'ютерна інженерія»

Цуркана Максима Сергійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: \_\_\_\_\_

Кількість балів: \_\_\_\_\_

ECTS: \_\_\_\_\_

Дата захисту: \_\_\_\_\_

**ОДЕСА – 2026**

## **ЗМІСТ**

|                                     |           |
|-------------------------------------|-----------|
| <b>ВСТУП</b>                        | <b>3</b>  |
| <b>ЗАВДАННЯ</b>                     | <b>4</b>  |
| <b>1. ТЕОРЕТИЧНА ЧАСТИНА</b>        | <b>5</b>  |
| 1.1 Поняття ООП та успадкування     | 5         |
| 1.2 Опис об'єктної частини завдання | 6         |
| <b>2. ПРАКТИЧНА ЧАСТИНА</b>         | <b>7</b>  |
| 2.1 Опис класів та ієрархії         | 7         |
| 2.2 Інтерфейс програми              | 12        |
| <b>ВИСНОВКИ</b>                     | <b>16</b> |
| <b>СПИСОК ЛІТЕРАТУРИ</b>            | <b>17</b> |

## **ВСТУП**

Мета роботи – реалізувати ієрархію класів на тему «Система управління персоналом» та застосувати її для розв'язання прикладної задачі, визначеної у варіанті завдання.

Під час виконання роботи необхідно використовувати основні принципи об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно із завданням потрібно реалізувати систему взаємопов'язаних класів, що описують персонал підприємства.

Необхідно розробити базовий клас `Person` з відповідними полями та методами, а також два класи-спадкоємці: `Worker` (Робітник) та `Electrician` (Електрик). Клас `Electrician` використовує додатковий клас `Object`, що не є спадкоємцем `Person`. Поля класів мають бути закритими (або захищеними), а доступ до них здійснюється через методи-аксесори.

Програма реалізує систему управління персоналом з можливістю додавання, видалення та виведення інформації про екземпляри типу Person та його спадкоємців через текстове інтерактивне меню. Для зберігання об'єктів використовується вектор вказівників на Person.

## ЗАВДАННЯ

Варіант 19. Система управління персоналом.

**Таблиця 1. Клас Person (варіант 19)**

| №  | Поля                                                                                                        | Методи                                              |
|----|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| 19 | ПІБ fullName<br>зріст height<br>вага weight<br>стать gender<br>країна проживання country<br>ел. пошта email | 1. має зріст понад 200 см?<br>2. чи живе в Україні? |

**Таблиця 2. Варіанти класів спадкоємців (варіант 19)**

| Варіант за<br>списком<br>групи | Обов'язковий<br>спадкоємець | Додатковий спадкоємець |
|--------------------------------|-----------------------------|------------------------|
| 19                             | Робітник                    | Електрик               |

5. Робітник: додаткові поля: завод, посада, оклад. Метод збільшити оклад, який збільшує оклад робітника залежно від посади. Також реалізувати метод – змінити посаду.

7. Електрик: стаж, розряд та вказівник на об'єкт, на якому він зараз працюватиме (клас Object, який не є спадкоємцем Person). Клас Object зберігає мінімально необхідний стаж та розряд для роботи на об'єкті. Метод перевіряє, чи допускається електрик на об'єкт (залежно від співвідношення стажу та розряду електрика та об'єкта). Метод змінити об'єкт із параметром – вказівник на новий об'єкт. Для заданого об'єкта перерахувати всіх електриків, які можуть працювати.

## 1. ТЕОРЕТИЧНА ЧАСТИНА

### 1.1 Поняття ООП та успадкування

Об'єктно-орієнтоване програмування (ООП) – це метод програмування, заснований на поданні програми у вигляді сукупності взаємодіючих об'єктів, кожен з яких є екземпляром певного класу. Класи утворюють ієрархії спадкування, що дозволяють будувати складні системи з компонентів, які повторно використовуються [1].

Основними принципами ООП є [2, 3]:

- а) Інкапсуляція – механізм, який поєднує дані та методи, що обробляють ці дані, і захищає їх від зовнішнього некоректного впливу. В C++ реалізується за допомогою специфікаторів доступу private, protected та public.
- б) Наслідування – механізм, за допомогою якого новий клас (спадкоємець) набуває властивостей та поведінки існуючого класу (базового). У даній роботі Worker та Electrician успадковують від Person.
- в) Поліморфізм – здатність об'єктів різних класів відповідати на одне й те ж саме повідомлення кожен по-своєму. Реалізується через віртуальні методи (virtual) та перевизначення (override).

Ключовим інструментом для реалізації динамічного поліморфізму в C++ є ключове слово `virtual`. Якщо метод оголошено як `virtual` у базовому класі, то при виклику через вказівник або посилання на базовий клас буде викликано реалізацію з фактичного класу об'єкта.

Перевизначення оператора вставки в потік (`operator<<`) є зручним механізмом для виведення об'єктів у стандартний вивід. В даній роботі кожен клас перевизначає `operator<<` та метод `to_string()`, що повертає рядкове представлення об'єкта.

## 1.2 Опис об'єктної частини завдання

Система управління персоналом будується навколо базового класу `Person`, який містить загальні атрибути особи: ПІБ, зріст, вагу, стать, країну проживання та електронну пошту. Клас надає два основні методи-предикати: перевірку зросту понад 200 см та перевірку проживання в Україні.

Клас `Worker` (Робітник) розширює `Person` додатковими полями, що описують трудову діяльність: завод, посада та оклад. Метод `increaseSalary()` реалізує підвищення окладу залежно від посади – від 30% для директора до 5% для інших посад. Метод `changePosition()` дозволяє змінити посаду.

Клас `Electrician` (Електрик) також є спадкоємцем `Person` і зберігає стаж роботи та розряд. Через вказівник зберігається поточний об'єкт (підстанція, агрегат тощо), на якому електрик працює. Допоміжний клас `Object` (не є спадкоємцем `Person`) зберігає назву об'єкта та мінімальні вимоги до стажу й розряду для роботи. Метод `canWorkOnObject()` перевіряє відповідність кваліфікації, метод `changeObject()` змінює призначення з перевіркою допуску.

Клас `PersonSystem` реалізує сховище об'єктів у вигляді вектора унікальних вказівників `std::unique_ptr<Person>`. Це дозволяє зберігати об'єкти різних типів (`Person`, `Worker`, `Electrician`) та звертатися до них поліморфно. Метод `printElectriciansForObject()` виводить усіх електриків, що відповідають вимогам конкретного об'єкта.

## 2. ПРАКТИЧНА ЧАСТИНА

### 2.1 Опис класів та ієрархії

Нижче наведено структуру всіх класів проекту.

1) Клас «Person» (базовий):

- d) Захищені поля: `fullName` (ПІБ), `height` (зріст), `weight` (вага), `gender` (стать), `country` (країна), `email` (ел. пошта).
- e) Конструктори: без параметрів, з параметрами, копіювання.
- f) Методи: `isTallerThan200()` – повертає `true` якщо зріст  $> 200$  см;  
`livesInUkraine()` – повертає `true` якщо країна «Україна».
- g) Перевизначення `to_string()` та `operator<<` – виводять інформацію про об'єкт.
- h) Перевантажені оператори: `operator=` (присвоєння), `operator==` (рівність за ПІБ та email).

2) Клас «Worker» (спадкоємець Person):

- a) Захищені поля: `factory` (завод), `position` (посада), `salary` (оклад).
- b) Конструктори: без параметрів, з параметрами, копіювання.
- c) Метод `increaseSalary()` – збільшує оклад залежно від посади (директор +30%, менеджер +20%, інженер +15%, технік +10%, інші +5%).
- d) Метод `changePosition()` – змінює посаду.
- e) Перевизначення `to_string()` та `operator<<`.

3) Допоміжний клас «Object»:

- a) Приватні поля: `name` (назва об'єкта), `minExperience` (мін. стаж), `minGrade` (мін. розряд).
- b) Конструктори: без параметрів, з параметрами, копіювання.

с) Повний набір гетерів та сетерів. Метод `to_string()` та `operator<<`.

#### 4) Клас «Electrician» (спадкоємець Person):

- а) Приватні поля: `experience` (стаж, роки), `grade` (розряд), `currentObject` (вказівник на Object).
- б) Конструктори: без параметрів, з параметрами (необов'язковий параметр `obj=nullptr`), копіювання.
- с) Метод `canWorkOnObject()` (без параметрів) – перевіряє допуск до поточного об'єкта.
- д) Метод `canWorkOnObject(const Object&)` – перевіряє допуск до довільного об'єкта.
- е) Метод `changeObject(Object*)` – призначає новий об'єкт з перевіркою допуску, повертає `bool`.
- ф) Перевизначення `to_string()` та `operator<<`.

#### 5) Клас «PersonSystem»:

- а) Приватні поля: `people` – вектор `std::unique_ptr<Person>`; `objects` – вектор Object.
- б) Методи додавання: `addPerson()`, `addWorker()`, `addElectrician()`.
- с) Методи видалення: `removeByName()`, `removeByIndex()`.
- д) Методи виведення: `printAll()`, `printWorkers()`, `printElectricians()`.
- е) Методи пошуку: `findByName()`, `findIndex()`, `findByType()`.
- ф) Методи для об'єктів: `addObject()`, `findObject()`, `printObjects()`, `printElectriciansForObject()`.

#### Лістинг класів

```
#pragma once
#include <iostream>
#include <string>
#include <sstream>
```

```

class Person {
protected:
    std::string fullName;
    int height;
    double weight;
    std::string gender;
    std::string country;
    std::string email;
public:
    Person();
    Person(const std::string& fullName, int height, double
weight,
           const std::string& gender, const std::string&
country,
           const std::string& email);
    Person(const Person& other);
    virtual ~Person() = default;

    std::string getFullName() const { return fullName; }
    int getHeight() const { return height; }
    double getWeight() const { return weight; }
    std::string getGender() const { return gender; }
    std::string getCountry() const { return country; }
    std::string getEmail() const { return email; }

    void setFullName(const std::string& v) { fullName = v;
}
    void setHeight (int v) { height = v; }
    void setWeight (double v) { weight = v; }
    void setGender (const std::string& v) { gender = v; }
    void setCountry(const std::string& v) { country = v; }
    void setEmail (const std::string& v) { email = v; }

    bool isTallerThan200() const;
    bool livesInUkraine() const;

    virtual std::string to_string() const;
    virtual void print() const;
    virtual std::string type() const { return "Person"; }

    friend std::ostream& operator<<(std::ostream& os, const
Person& p);
    Person& operator=(const Person& other);
    bool operator==(const Person& other) const;
};

```

**Лістинг 1 – Оголошення класу Person (Person.h)**



```

#pragma once
#include "Person.h"

class Worker : public Person {
protected:
    std::string factory;
    std::string position;
    double salary;
public:
    Worker();
    Worker(const std::string& fullName, int height, double
weight,
           const std::string& gender, const std::string&
country,
           const std::string& email,
           const std::string& factory,
           const std::string& position, double salary);
    Worker(const Worker& other);
    virtual ~Worker() = default;

    std::string getFactory() const { return factory; }
    std::string getPosition() const { return position; }
    double getSalary() const { return salary; }

    void setFactory (const std::string& v) { factory = v; }
    void setPosition(const std::string& v) { position = v; }
}
    void setSalary (double v) { salary = v; }

    void increaseSalary();
    void changePosition(const std::string& newPosition);

    std::string to_string() const override;
    void print() const override;
    std::string type() const override { return "Worker"; }

    friend std::ostream& operator<<(std::ostream& os, const
Worker& w);
};

```

## Лістинг 2 – Оголошення класу Worker (Worker.h)

```

#pragma once
#include "Person.h"

class Object {
    std::string name;
    int minExperience;

```

```

        int minGrade;
public:
    Object();
    Object(const std::string& name, int minExperience, int
minGrade);
    Object(const Object& other);

    std::string getName()          const { return name; }
    int getMinExperience()         const { return
minExperience; }
    int getMinGrade()             const { return minGrade;
}

    void setName(const std::string& v) { name = v; }
    void setMinExperience(int v)      { minExperience =
v; }
    void setMinGrade(int v)          { minGrade = v; }

    std::string to_string() const;
    friend std::ostream& operator<<(std::ostream& os, const
Object& o);
};

class Electrician : public Person {
    int experience;
    int grade;
    Object* currentObject;
public:
    Electrician();
    Electrician(const std::string& fullName, int height,
double weight,
                const std::string& gender, const
std::string& country,
                const std::string& email,
                int experience, int grade, Object* obj =
nullptr);
    Electrician(const Electrician& other);
    ~Electrician() override = default;

    int getExperience()    const { return experience; }
    int getGrade()        const { return grade; }
    Object* getCurrentObject() const { return
currentObject; }

    void setExperience(int v)    { experience = v; }
    void setGrade(int v)        { grade = v; }
    void setCurrentObject(Object* o) { currentObject = o; }

    bool canWorkOnObject() const;
    bool canWorkOnObject(const Object& obj) const;

```

```

    bool changeObject(Object* newObj);

    std::string to_string() const override;
    void print() const override;
    std::string type() const override { return
"Electrician"; }

    friend std::ostream& operator<<(std::ostream& os, const
Electrician& e);
};

```

### Лістинг 3 – Оголошення класів Object та Electrician (Electrician.h)

```

#include "../include/Person.h"

Person::Person()
    : fullName(""), height(0), weight(0.0),
      gender(""), country(""), email("") {}

Person::Person(const std::string& fullName, int height,
double weight,
               const std::string& gender, const
std::string& country,
               const std::string& email)
    : fullName(fullName), height(height), weight(weight),
      gender(gender), country(country), email(email) {}

Person::Person(const Person& other)
    : fullName(other.fullName), height(other.height),
      weight(other.weight), gender(other.gender),
      country(other.country), email(other.email) {}

Person& Person::operator=(const Person& other) {
    if (this != &other) {
        fullName = other.fullName;
        height    = other.height;
        weight    = other.weight;
        gender    = other.gender;
        country   = other.country;
        email     = other.email;
    }
    return *this;
}

bool Person::operator==(const Person& other) const {
    return fullName == other.fullName && email ==
other.email;
}

```

```

std::string Person::to_string() const {
    std::ostringstream oss;
    oss << "[Person]\n"
        << "    ПІБ      : " << fullName << "\n"
        << "    Зріст   : " << height   << " см\n"
        << "    Вага    : " << weight   << " кг\n"
        << "    Стать  : " << gender   << "\n"
        << "    Країна  : " << country  << "\n"
        << "    Email   : " << email    << "\n"
        << "    >200 см : " << (isTallerThan200() ? "так" :
"ні") << "\n"
        << "    Україна : " << (livesInUkraine() ? "так" :
"ні") << "\n";
    return oss.str();
}

std::ostream& operator<<(std::ostream& os, const Person& p)
{
    return os << p.to_string();
}

```

#### ЛІСТИНГ 4 – Реалізація класу Person (Person.cpp)

```

#include "../include/Worker.h"

Worker::Worker()
    : Person(), factory(""), position(""), salary(0.0) {}

Worker::Worker(const std::string& fullName, int height,
               double weight, const std::string& gender,
               const std::string& country,
               const std::string& email,
               const std::string& factory,
               const std::string& position, double salary)
    : Person(fullName, height, weight, gender, country,
email),
      factory(factory), position(position), salary(salary)
{}

Worker::Worker(const Worker& other)
    : Person(other),
      factory(other.factory), position(other.position),
      salary(other.salary) {}

void Worker::increaseSalary() {
    if (position == "директор") salary *= 1.30;
    else if (position == "менеджер") salary *= 1.20;
}

```

```

        else if (position == "інженер")    salary *= 1.15;
        else if (position == "технік")     salary *= 1.10;
        else                               salary *= 1.05;
    }

    std::string Worker::to_string() const {
        std::ostringstream oss;
        oss << "[Worker]\n"
            << "    ПІБ      : " << fullName << "\n"
            << "    Завод   : " << factory   << "\n"
            << "    Посада  : " << position  << "\n"
            << "    Оклад   : " << salary    << " грн\n";
        return oss.str();
    }

    std::ostream& operator<<(std::ostream& os, const Worker& w)
    {
        return os << w.to_string();
    }

```

### Лістинг 5 – Реалізація класу Worker (Worker.cpp)

```

#include "../include/Electrician.h"
#include <sstream>

bool Electrician::canWorkOnObject() const {
    if (!currentObject) return false;
    return canWorkOnObject(*currentObject);
}

bool Electrician::canWorkOnObject(const Object& obj) const
{
    return experience >= obj.getMinExperience() &&
           grade      >= obj.getMinGrade();
}

bool Electrician::changeObject(Object* newObj) {
    if (newObj && !canWorkOnObject(*newObj)) return false;
    currentObject = newObj;
    return true;
}

std::string Electrician::to_string() const {
    std::ostringstream oss;
    oss << "[Electrician]\n"
        << "    ПІБ      : " << fullName   << "\n"
        << "    Стаж     : " << experience << " р.\n"
        << "    Розряд  : " << grade      << "\n";
}

```

```

        << "  Об'єкт : ";
    if (currentObject)
        oss << currentObject->getName()
            << " (допуск: "
            << (canWorkOnObject() ? "так" : "ні") << ")";
    else
        oss << "не призначено";
    oss << "\n";
    return oss.str();
}

```

Лістинг 6 – Реалізація ключових методів Electrician (Electrician.cpp)

## 2.2 Інтерфейс програми

Програма реалізує інтерактивне текстове меню, яке дозволяє користувачу обирати операції у довільному порядку.

Головне меню програми (рис. 2.1) містить 16 пунктів:

| СИСТЕМА УПРАВЛІННЯ ПЕРСОНАЛОМ |                               |
|-------------------------------|-------------------------------|
| 1.                            | Додати Person                 |
| 2.                            | Додати Worker (Робітник)      |
| 3.                            | Додати Electrician (Електрик) |
| 4.                            | Видалити за іменем            |
| 5.                            | Видалити за індексом          |
| 6.                            | Вивести всіх                  |
| 7.                            | Вивести лише Workers          |
| 8.                            | Вивести лише Electricians     |
| 9.                            | Пошук за іменем               |
| 10.                           | Збільшити оклад робітника     |
| 11.                           | Змінити посаду робітника      |
| 12.                           | Перевірити допуск електрика   |
| 13.                           | Змінити об'єкт електрика      |
| 14.                           | Додати об'єкт (довідник)      |
| 15.                           | Вивести об'єкти               |
| 16.                           | Електрики для об'єкта         |
| 0.                            | Вихід                         |

Оберіть дію:

Рисунок 2.1 – Головне меню програми

При виборі пунктів 1–3 користувачу пропонується послідовно ввести всі поля нового об'єкта. Для Worker та Electrician спочатку вводяться поля базового класу Person, а потім додаткові поля спадкоємця.

Пункти 4 та 5 дозволяють видалити запис – за точним ПІБ або за порядковим номером у списку (індексація з 0).

Пункти 6–8 виводять усі записи або відфільтровані за типом. Вивід використовує поліморфний метод print(), що викликає перевизначену версію to\_string() відповідного класу.

Пункт 9 здійснює пошук за повним ПІБ та виводить детальну інформацію про знайдений запис.

Пункти 10–11 призначені для операцій над робітниками: збільшення окладу (відповідно до посади) та зміна посади. Пункти 12–13 – для операцій над електриками: перевірка допуску до поточного об'єкта та зміна об'єкта (з перевіркою відповідності вимогам).

Пункти 14–16 забезпечують роботу з довідником об'єктів: додавання нового об'єкта, виведення всіх об'єктів та пошук електриків, що відповідають вимогам конкретного об'єкта.

Приклад виведення окремих записів:

```
[Person]
ПІБ      : Коваль Іван Петрович
Зріст    : 185 см
Вага     : 80 кг
Стать    : чол
Країна   : Україна
Email    : koval@mail.ua
>200 см  : ні
Україна  : так

[Worker]
ПІБ      : Мельник Олег Сергійович
Зріст    : 175 см
...
Завод    : Завод №1
```

Посада : інженер  
Оклад : 25000 грн

[Electrician]  
ПІВ : Бондар Микола Олексійович  
...  
Стаж : 4 р.  
Розряд : 5  
Об'єкт : Підстанція А (допуск: так)

## Рисунок 2.2 – Приклад виводу об'єктів різних типів

```
int main() {
#ifdef _WIN32
    system("chcp 65001 > nul");
#endif
    PersonSystem sys;

    // Додавання тестових об'єктів
    sys.addObject(Object("Підстанція А", 3, 4));
    sys.addObject(Object("Підстанція В", 5, 5));

    // Тестові записи
    sys.addPerson(Person("Коваль Іван Петрович", 185, 80.0,
                        "чол", "Україна",
                        "koval@mail.ua"));
    sys.addWorker(Worker("Мельник Олег Сергійович", 175,
                        75.0,
                        "чол", "Україна",
                        "melnik@mail.ua",
                        "Завод №1", "інженер", 25000.0));

    Object* objA = sys.findObject("Підстанція А");
    sys.addElectrician(Electrician("Бондар Микола", 178,
                        82.0,
                        "чол", "Україна",
                        "bondar@mail.ua", 4, 5,
                        objA));

    // Інтерактивне меню
    int choice = -1;
    while (choice != 0) {
        showMenu();
        std::cin >> choice;
        clearInput();
        // ... обробка вибору ...
    }
    return 0;
}
```



```
}
```

Лістинг 7 – Фрагмент функції `main()` з ініціалізацією та циклом меню

## ВИСНОВКИ

У ході виконання лабораторної роботи було розроблено програму на мові C++ з використанням принципів об'єктно-орієнтованого програмування на тему «Система управління персоналом».

В процесі роботи була реалізована ієрархія класів: базовий клас `Person`, обов'язковий спадкоємець `Worker` та додатковий спадкоємець `Electrician`. Додатково розроблено допоміжний клас `Object`, що не входить до ієрархії `Person` і використовується для зберігання вимог до кваліфікації електриків.

При розробці програми були застосовані всі основні принципи ООП:

- інкапсуляція – поля класів оголошено як `protected` або `private`, доступ здійснюється через гетери/сетери;
- наслідування – `Worker` та `Electrician` успадковують усі поля та методи `Person`, розширюючи їх;
- поліморфізм – метод `print()` оголошено як `virtual`, перевизначено в кожному класі; вектор вказівників на `Person` дозволяє поліморфно обробляти об'єкти різних типів.

Для зберігання колекції об'єктів використовується вектор `std::vector<std::unique_ptr<Person>>`, що забезпечує автоматичне управління пам'яттю. Реалізовано повний набір операцій: додавання, видалення за ім'ям та індексом, виведення всіх або відфільтрованих записів, пошук, а також специфічні операції над спадкоємцями.

У результаті виконання лабораторної роботи були закріплені практичні навички програмування на C++, роботи з класами, спадкуванням, поліморфізмом, колекціями, вказівниками та консольним інтерфейсом.

## СПИСОК ЛІТЕРАТУРИ

1. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oop>
2. W3School, C++ OOP (Object-Oriented Programming). Режим доступу: [https://www.w3schools.com/cpp/cpp\\_oop.asp](https://www.w3schools.com/cpp/cpp_oop.asp)
3. Є.О. Віктор, О.А. Геренко, І.М. Шпінарева, Практикум з курсу Об'єктно-орієнтоване програмування мовою C#. [Електронне видання] / Одеса, 2026. – Режим доступу: <https://oop-practicum-csharp-851f60.gitlab.io/>
4. Репозиторій GitHub лабораторної роботи. – Режим доступу: [https://github.com/Offiks/lab4\\_PersonSystem](https://github.com/Offiks/lab4_PersonSystem)